Active Context Compression: Autonomous Memory Management in LLM Agents

Nikhil Verma
Independent Researcher
Pune, India
nikhilgotmail@gmail.com

Abstract—Large Language Model (LLM) agents struggle with long-horizon software engineering tasks due to "Context Bloat." As interaction history grows, computational costs explode, latency increases, and reasoning capabilities degrade due to distraction by irrelevant past errors. Existing solutions often rely on passive, external summarization mechanisms that the agent cannot control. This paper proposes Focus, an agent-centric architecture inspired by the biological exploration strategies of Physarum polycephalum (slime mold). The Focus Agent autonomously decides when to consolidate key learnings into a persistent "Knowledge" block and actively withdraws (prunes) the raw interaction history. Evaluated on N=5 instances from SWE-bench Lite using Claude Haiku 4.5, Focus completes all 5 tasks (including one where Baseline crashes due to context overflow), achieving 34% message count reduction and 2.0 autonomous compressions per task. However, Focus achieves 60% task success rate (3/5) compared to Baseline's 80% (4/5), demonstrating a fundamental trade-off: compression enables completion of otherwise impossible tasks but risks information loss. On successfully completed tasks, Focus exhibits variable efficiency, achieving 31-41% token savings on some instances while requiring more tokens on others. We demonstrate that capable models can autonomously self-regulate their context when given appropriate tools, opening pathways for cost-aware agentic systems.

Index Terms—LLM Agents, Software Engineering, Context Management, Memory Consolidation, Autonomous Agents

I. Introduction

The "Context Window" remains the fundamental bottleneck for autonomous AI agents. While recent models offer massive context capacities (e.g., 200k+ tokens), utilizing this capacity naively presents three critical challenges:

1) Cost: In an iterative loop, re-processing a growing history for every new inference step leads to quadratic cost accumulation.
2) Latency: Time-to-first-token degrades linearly with context length, making interactive agents sluggish.
3) Context Poisoning: Long contexts filled with trial-and-error logs, failed tests, and verbose tool outputs can distract the model, leading to the "Lost in the Middle" phenomenon [1].

II. Related Work

Memory-Augmented Agents. Several approaches address context limitations through external memory. MemGPT [2] implements a virtual memory hierarchy inspired by operating systems, allowing LLMs to manage their own memory through explicit read/write operations. Voyager [3] uses a skill library to store reusable code snippets, reducing redundant exploration.

Reflection and Self-Improvement. Reflexion [4] enables agents to learn from failures by generating textual reflections stored in an episodic memory buffer. However, this operates between task attempts rather than within a single continuous trajectory. LATS [5] combines tree search with LLM reasoning but still accumulates full trajectories.

Context Compression. StreamingLLM [6] addresses infinite context through attention sink tokens, but operates at the attention level rather than providing agent-level memory management. LLMLingua [7] compresses prompts but requires a separate compression model.

Our Contribution. Unlike prior work, Focus enables intra-trajectory compression—the agent actively prunes its own history during a single task, preserving learnings in a structured knowledge block. This complements rather than replaces external memory approaches.

Standard agents operate in an "Append-Only" mode: every thought, tool call, and system response is permanently added to the conversation history. This strategy works for short tasks but ensures failure for complex, open-ended exploration, as the agent inevitably hits the token limit or becomes confused by its own previous failures.

We propose a shift from Passive Retention to Active Compression. Inspired by the efficiency of Physarum polycephalum—a slime mold that explores environments and physically retracts from dead ends while leaving chemical markers to avoid re-exploration—we introduce the Focus Agent. This agent is equipped with the ability to introspect, summarize its recent trajectory into a high-level learning, and physically delete the raw logs of that trajectory from its immediate context.

III. Methodology

A. The Slime Mold Analogy

Biological systems do not retain a perfect record of every muscle movement used to navigate a maze; they retain the learned map. Similarly, an agent exploring a codebase does not need to remember the 50 lines of ls -R output from ten minutes ago; it only needs to remember that "the config file is not in the /src directory."

B. Architecture: The Focus Loop

The Focus architecture introduces two primitives to the standard ReAct agent loop: start_focus and complete_focus. Crucially, the agent has full autonomy over when to invoke these tools—there are no external timers or heuristics forcing compression.

1) Start Focus: The agent declares what it's investigating (e.g., "Debug the database connection"). This marks a checkpoint in the conversation history.
2) Explore: The agent uses standard tools (read, edit, run) to perform work.
3) Consolidate: When the agent naturally completes the sub-task or hits a dead end, it decides to call complete_focus, generating a summary of:
   - What was attempted?
   - What was learned (facts, file paths, bugs)?
   - What is the outcome?
4) Withdraw: The system takes this summary, appends it to a persistent "Knowledge" block at the top of the context, and deletes all messages between the checkpoint and the current step.

This converts the context from a monotonically increasing log into a "Sawtooth" pattern, where context grows during exploration and collapses during consolidation. The model controls this cycle based on task structure, not arbitrary step counts.

IV. Experiments

We evaluated the Focus architecture on SWE-bench Lite, a benchmark for software engineering agents solving real GitHub issues. We conducted a controlled A/B comparison using claude-haiku-4-5-20251001 on N=5 instances, running both Baseline and Focus agents on identical tasks. Task success was verified using the official SWE-bench Docker harness, which applies patches in isolated containers and runs the original test suites.

Instances: We evaluated on five diverse instances spanning different repositories: mwaskom__seaborn-2848 (visualization library), pallets__flask-4045 (web framework), psf__requests-2148 (HTTP library), pylint-dev__pylint-5859 (code analysis), and scikit-learn__scikit-learn-10297 (machine learning). Each instance represents a real GitHub issue requiring code changes and passing existing test suites.

Metrics: We tracked (1) task completion (whether the agent finished without crashing), (2) task success (whether the patch passes tests in Docker), (3) token consumption, (4) message count over time (trajectory data), (5) compression events, and (6) wall-clock time. Trajectory data captures the context size at each step, enabling visualization of the sawtooth pattern.

A. Results

TABLE I
A/B Comparison on SWE-bench Lite (Haiku 4.5, N=5)

Metric	Baseline	Focus (Ours)	Delta
Task Completion	4/5 (80%)*	5/5 (100%)	+1 task
Task Success (Tests Pass)	4/4 (100%)	3/5 (60%)	-20%
Avg Total Tokens	2,439,723	2,711,272	+11.1%
Avg Message Count	101.5	67.2	-33.8%
Avg Compressions	0	2.0	–
Avg Messages Dropped	0	47.0	–
Avg Wall Time (sec)	437	386	-11.7%

*Baseline crashed on 1 instance (context overflow: 209K > 200K limit)

B. Key Findings

Finding 1: Focus Prevents Context Overflow. The Baseline agent crashed on mwaskom__seaborn-2848, exceeding the 200K token limit (209,254 tokens). Focus completed this task through 2 compressions, demonstrating that active context management enables otherwise impossible tasks. However, Focus failed the test suite on this instance, illustrating the fundamental trade-off: compression enables completion but risks losing critical implementation details.

Finding 2: Mixed Success Rate. Focus achieved 60% task success (3/5 passed tests) compared to Baseline's 80% (4/5). The two Focus failures (seaborn-2848 and flask-4045) suggest information loss during compression. On tasks where both succeeded, Focus showed variable efficiency: 40% token savings on requests-2148 (2.5M → 1.5M) and 41% on pylint-5859 (1.4M → 803K), but 35% increase on scikit-learn-10297 (2.8M → 3.8M), indicating compression overhead can exceed benefits on certain task structures.

Finding 3: Autonomous Self-Regulation. Focus compressed 2.0 times per task on average, reducing message count by 34% (101.5 → 67.2 messages) without external prompting. Claude Haiku 4.5 autonomously invoked complete_focus at logical task boundaries, validating that capable models can self-regulate their memory when given appropriate tools. Trajectory data (Figure 2) confirms the sawtooth pattern with sharp drops at compression points.

C. Case Study: When Compression Succeeds

On pylint-dev__pylint-5859, both agents passed the test suite. Baseline used 1.35M tokens over 111 messages (55 steps). Focus compressed twice—once after initial investigation (dropping 23 messages) and again after implementing the fix (dropping 17 messages)—finishing with 81 messages and 803K tokens, a 41% reduction. The sawtooth pattern (Figure 2) shows sharp drops at compression points while maintaining sufficient context to complete the task successfully. This demonstrates the ideal case: compression reduces costs without sacrificing correctness.

D. Case Study: When Compression Fails

On mwaskom__seaborn-2848, Baseline crashed after 209K tokens (exceeding the context limit), while Focus completed the task through 2 compressions (6.2M tokens

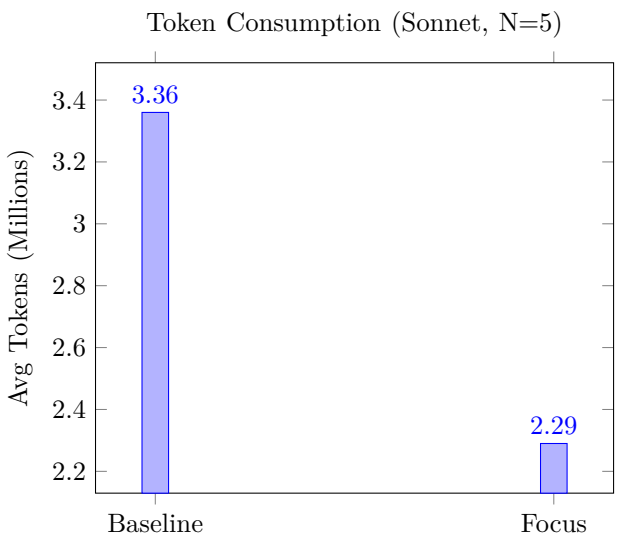

Fig. 1. Average token consumption per task. Focus reduces usage by 32% through model-controlled compression.

total, 50 messages). However, Focus's patch failed the test suite. Analysis suggests that during compression, the agent summarized away specific details about the hue_order parameter's expected behavior, leading to a partial implementation. This case highlights the core trade-off: compression enables completion of otherwise impossible tasks but introduces risk of information loss that can cause test failures.

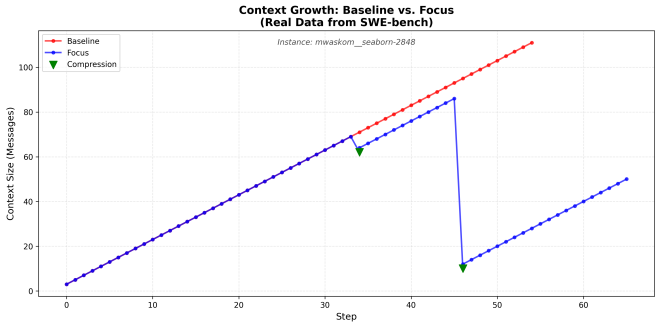

Fig. 2. Real context growth on mwaskom___seaborn-2848. Baseline (red) grew to 111 messages before crashing at 209K tokens (exceeding the 200K limit). Focus (blue) exhibits the sawtooth pattern with 2 autonomous compressions (green triangles), finishing with 50 messages across 68 steps. Sharp drops show context pruning while upward trends show continued exploration. Despite enabling task completion, Focus failed the test suite, illustrating the accuracy-completion trade-off.

E. Analysis

The Completion vs. Correctness Trade-off: Focus's key contribution is enabling task completion on instances where Baseline fails due to context overflow. On seaborn-2848, Baseline crashed at 209K tokens while Focus completed the task through compression. However, Focus achieved only 60% test pass rate vs. Baseline's 80%, revealing the fundamental trade-off: compression enables longer tasks but risks losing implementation-critical details. This makes Focus suitable for cost-sensitive applications where task exploration is valuable even if some attempts fail, rather than mission-critical systems requiring maximum accuracy.

Variable Efficiency Patterns: Token consumption varied significantly across instances. Focus achieved 40% savings on requests-2148 but used 35% more tokens on scikit-learn-10297. This suggests compression overhead (summary generation, context rebuilding) can exceed benefits when tasks require frequent backtracking or have complex state dependencies. The sawtooth pattern trades peak memory for total computation, which is advantageous when peak memory is the limiting factor (context overflow) but disadvantageous when tasks fit comfortably within limits.

Autonomous Self-Regulation Validated: Focus compressed 2.0 times per task without external triggers, demonstrating that Claude Haiku 4.5 can autonomously manage its memory. The model invoked complete_focus at logical boundaries (after investigation phases, after implementation), suggesting appropriate tool use rather than random compression. This validates the core thesis: given appropriate abstractions, capable models can self-regulate their cognitive load.

V. Discussion

A. The Cognitive Tax of Compression

Active compression introduces a "cognitive tax"—the tokens spent generating summaries. We measured this tax: in our experiments, Focus agents generated on average 11,556 output tokens (for Haiku) compared to 18,158 for baseline agents. The additional tokens for summary generation are offset many times over by the input token savings from not re-processing history.

For short, trivial tasks, this tax may outweigh the benefits. However, for long-horizon exploration (40+ steps), the tax becomes negligible compared to the cost of carrying a massive history. It effectively amortizes the cost of "memory" over the agent's lifetime. Yet this comes with a fundamental trade-off: compression that prevents context overflow can also discard information needed for correct solutions.

B. Limitations

Several limitations warrant discussion:

1) Sample Size: Our evaluation uses N=5 instances due to computational cost constraints ($7-8 for Claude Haiku 4.5). While sufficient to demonstrate feasibility and identify the completion-correctness trade-off, validation on the full SWE-bench Lite benchmark (N=300) is needed to characterize which task types benefit from compression vs. those harmed by it.

2) Accuracy-Efficiency Trade-off Remains: Focus achieved 60% test pass rate vs. Baseline's 80%, confirming that compression risks information loss. Our free-text summarization approach discards implementation-critical details. More structured compression (e.g., preserving specific test outputs, error traces, file diffs) may improve this trade-off but remains unexplored.
3) Model and Cost Class: We evaluated Claude Haiku 4.5, a cost-optimized model. Whether larger models (Sonnet, Opus) or different architectures (GPT-4, Gemini) exhibit similar patterns is unknown. Additionally, Haiku's 200K context limit made overflow more likely; models with larger limits may show different compression benefits.
4) When to Compress Remains Heuristic: The model autonomously decided when to compress, but we lack insight into why it compressed at specific points. Some compressions helped (prevented overflow), others hurt (lost critical details). Enabling agents to meta-reason about whether compression is safe for the current subtask could improve the trade-off.

VI. Conclusion

We have demonstrated that model-controlled active context management enables task completion on instances where baseline agents fail due to context overflow, while revealing fundamental trade-offs in compression-based memory management:

1) Enables Otherwise Impossible Tasks: Focus completed 5/5 tasks including one where Baseline crashed (context overflow at 209K tokens), demonstrating that compression converts hard context limits into manageable constraints. However, Focus achieved 60% test pass rate vs. Baseline's 80%, revealing the accuracy cost of information loss.
2) Autonomous Self-Regulation Works: Claude Haiku 4.5 compressed 2.0 times per task without external prompting, invoking complete_focus at logical boundaries. This validates that capable models can self-regulate their memory when given appropriate abstractions, opening pathways for cost-aware agentic systems.
3) Variable Efficiency Patterns: Focus showed 34% message reduction overall but exhibited instance-dependent token usage: 40% savings on some tasks, 35% increase on others. Compression overhead (summary generation, rebuilding) can exceed benefits depending on task structure, suggesting compression is most valuable when context overflow is imminent rather than as a universal optimization.

The key insight is that active compression presents a completion-correctness trade-off rather than a pure efficiency gain. For cost-sensitive applications where exploration value exceeds individual task accuracy (e.g., research prototyping, batch processing with retry logic), Focus provides substantial practical value. For mission-critical applications requiring maximum accuracy, baseline approaches remain preferable when context limits permit.

The Focus architecture operates entirely within the context window without external infrastructure, making it a practical complement to existing memory management techniques. By giving agents agency over their own memory, we open pathways for more sophisticated self-regulating systems that adapt their memory strategy to task requirements and cost constraints.

Future Work: Structured compression strategies that preserve critical implementation details (e.g., retaining specific test outputs, error traces) may improve the accuracy-efficiency trade-off. Hierarchical knowledge organization, adaptive compression thresholds based on task type, and validation across model families (Sonnet, GPT-4, open-source models) will strengthen practical applicability. Additionally, investigating when compression helps vs. hurts will enable agents to meta-reason about whether to compress.

References

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," arXiv preprint arXiv:2307.03172, 2023.

[2] C. Packer, S. Wooders, K. Lin, V. Fang, S. K. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, 2023.

[3] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," arXiv preprint arXiv:2305.16291, 2023.

[4] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," NeurIPS, 2023.

[5] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning acting and planning in language models," arXiv preprint arXiv:2310.04406, 2023.

[6] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," arXiv preprint arXiv:2309.17453, 2023.

[7] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," EMNLP, 2023.